固件解包工具

图文使用手册

单文件 · 零三方依赖 · Windows · Python 3.13+



适用版本: bare2.0 (含 fwtool 逆向打包)

涵盖 DD-5G / NRadio 固件全流程

目 录

第一章 工具概述

1.1 功能定位

1.2 运行环境

1.3 文件清单

第二章 安装与启动

2.1 Python 环境准备

2.2 CMD 双击启动器原理

2.3 首次启动验证

第三章 页签① 固件解包

3.1~3.6 分步解包

3.7 一键智能解包

第四章 页签② NR 备份

4.1 NR 查看/校验/解包

4.2 目录打包为 .nr

4.3 一键制作解锁备份

第五章 页签③ 逆向打包

5.1~5.4 cpio/tar/FS/拼接

5.5 ★ 分区→fwtool 升级包

第六章 CLI 命令行模式

第七章 实战: DD-5G 固件解包→修改→重打包

附录 A moses 段表参考

附录 B 常见问题

第一章 工具概述

1.1 功能定位

本工具是一个单文件 Python 程序（约 2300 行），以 Tkinter 构建三页签图形界面，同时提供 --cli 无界面模式。它将嵌入式路由器固件的解包、分析、修改、重打包全流程整合于一身，特别针对 OpenWrt fwtool 升级包和 NRadio .nr 备份格式做了深度适配。

核心能力:

· 固件分层识别与解包（gzip/cpio/JFFS2/squashfs/UBI 全链路）

· 厂商分段裸包按 upgrade.sh 段表自动切全分区

· NR (.nr) 备份文件的查看、校验、解包、修改、重打包、一键解锁

· 逆向打包: 从分区目录重建 fwtool 全量升级 .bin

· 纯 Python 实现 cpio newc 生成器、gzip 封帧、CRC32 计算，零三方依赖

1.2 运行环境

操作系统: Windows 10/11（PowerShell 环境）

Python: 3.10+（推荐 3.13，系统自带 tkinter）

可选依赖（解 JFFS2 时自动调用）：

```
python -m pip install --user jefferson
```

可选依赖（解 UBI 时自动调用）：

```
python -m pip install --user ubi_reader
```

提示: squashfs 解包需要 unsquashfs（WSL 或 7-Zip）；JFFS2/squashfs 重打包需要 mkfs.jffs2/mksquashfs（一般在 WSL 内执行）。

1.3 文件清单

文件名	说明
固件解包工具.py	主程序（单文件，含全部引擎与 GUI）

固件解包工具(双击启动).cmd	纯 ASCII 批处理启动器，自动查找并启动 .py
DD-5G_2.1.11.bin	示例固件（只读对照基准）

第二章 安装与启动

2.1 Python 环境准备

从 Microsoft Store 安装 Python 3.13，或从 python.org 下载。安装时勾选 "Add to PATH"。验证:

```
python --version
```

确认输出 Python 3.13.x 后，安装可选解包依赖:

```
python -m pip install --user jefferson
```

2.2 CMD 双击启动器原理

启动器文件名: 固件解包工具(双击启动).cmd

这是一个纯 ASCII 批处理脚本，设计目标是: 即使文件名或路径包含非 ASCII 字符（如中文"临时文件"），批处理本身也不会因编码问题出错。其工作原理如下:

第 1 步: 切换到脚本所在目录

```
cd /d "%~dp0"
```

第 2 步: 遍历当前目录所有 .py 文件，用 findstr 搜索内部标记 "firmware-unpack"，定位主程序文件名

```
for /f "delims=" %%F in ('dir /b *.py 2^>nul') do (
  findstr /c:"firmware-unpack" "%%F" >nul 2>&1 && set "GUIPY=%%F"
)
```

第 3 步: 用 python 启动; 若失败则回退到 py 启动器

```
python "%GUIPY%"
```

第 4 步: 程序退出后 pause 等待按键，防止窗口闪退

> *提示: 如果双击后窗口闪退，请检查: (1) Python 是否在 PATH 中; (2) 固件解包工具.py 是否与 .cmd 在同一目录; (3) 在 CMD 中手动运行 python 固件解包工具.py 查看报错。*

2.3 首次启动验证

双击 固件解包工具(双击启动).cmd，应出现标题为"固件 / NR 备份 可视化解包-打包工具 " 的窗口，包含三个页签:「固件解包」「NR 备份」「逆向打包」。



图 2-1 工具三页签界面布局示意

第三章 页签① 固件解包

本页签提供固件分层解包的完整工具链，从扫描识别到一键全自动解包，覆盖 firmware-unpack skill 的全部流程。

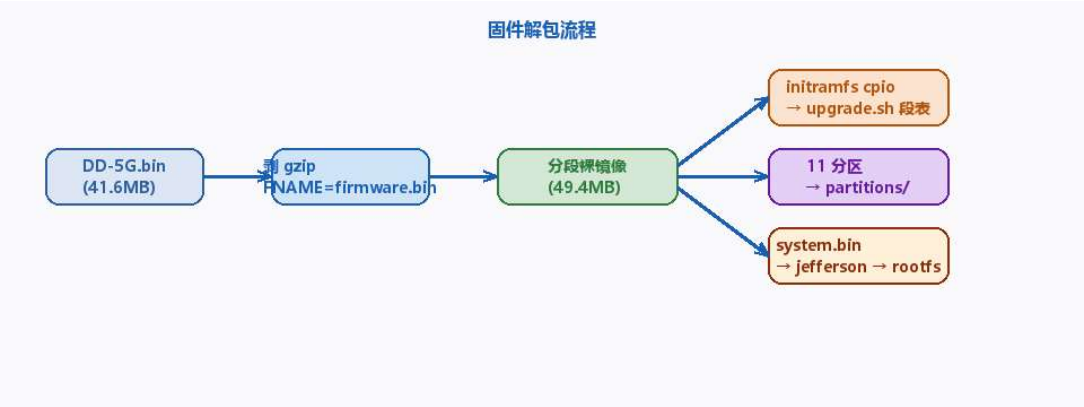


图 3-1 固件解包流程示意

3.1 扫描识别

[1 扫描识别] 按钮: 读取输入文件头部 256 字节的 hex+ASCII 快照、尾部 dump、全文件 magic 签名命中表（含偏移）、gzip 成员清单。自动判定偏移 0 的文件类型并给出下一步建议。

识别的 magic 类型: gzip/xz/zstd/bzip2/lzma-alone、JFFS2、squashfs、UBI、uImage、FIT/DTB、Android boot 等。

> *提示: 注意看尾部: OpenWrt sysupgrade 包常在压缩流尾部追加 JSON 元数据，扫描即可确认平台与版本，无需解包。*

3.2 剥一层压缩

[2 剥一层压缩] 按钮: 从指定偏移剥一个压缩成员（gzip/xz/lzma-alone/bzip2/zstd），输出解压大小和输入剩余字节数。

gzip 报 "input remaining > 0" 说明后面还有数据（分段镜像或第二个成员），不是错误。

3.3 解包 cpio

[3 解包 cpio] 按钮: 内嵌 newc(070701)/crc(070702) 解析器，支持串联成员，4 字节对齐。必须遇到 TRAILER!!! 才算成功，否则报错（防止巧合字节造成假成功）。

3.4 切分区段

[4 切分区段] 按钮: 按偏移/长度切片（支持 0x、K、M、G 写法），块大小+skip/count 一键填充。是厂商分段裸包的逆向操作。

3.5 解包文件系统

[5 解包文件系统] 按钮: 自动识别并调用对应解包器:

文件系统	解包方式	说明
JFFS2	jefferson (pip)	支持 zlib/RTIME/LZO 压缩节点
squashfs	unsquashfs / 7z	LZMA/ZSTD 需 sasquatch
UBI/UBIFS	ubireader (pip)	自动扫描 PEB/LEB
tar/zip	Python 内置	tarfile / zipfile
ext4	Python 内置	简易 ext4 reader

对分段裸镜像: 自动剥压缩外壳后递归判断文件系统，探测到 JFFS2 magic 自动切段再解。

3.6 找分区线索

[6 找分区线索] 按钮: 在固件内搜索 ptable/xloader/hiboot/kernel/system/teeos 等段名字符串，返回命中偏移以辅助推断段边界。

3.7 一键智能解包

[一键智能解包] 按钮: 完整自动化流程:

1) 逐层剥压缩（gzip → cpio-gzip → 裸镜像）

2) 偏移 0 无 magic 且体积 > 1MB → 按厂商分段裸包处理

3) 探测内部 gzip 成员（可实解验证，标记 cpio/kernel Image）

4) 从最大的 cpio-gzip（主 initramfs）内解析 upgrade.sh 的 fw_full_upgrade 段表（skip/count 按十进制）

5) 按段表切出全部 11 个分区 → 导出 partitions/ 目录

6) system 段用 jefferson 解出 rootfs

7) 按段跨度归类导出 teeos_cpio / kernel_Image / initramfs_cpio

提示: jefferson 遇到含特殊字符（如证书文件名中的 ő）会因 GBK 编码崩溃，工具已通过强制 PYTHONIOENCODING=utf-8 / PYTHONUTF8=1 环境变量修复。

提示: jefferson 要求目标目录不存在，工具自动 rmtree 后再调用; 退出码 4294967295 (unsigned -1) 是正常返回值。

第四章 页签② NR 备份

本页签针对 NRadio 设备 /sbin/sysupgrade 使用的 .nr 备份格式操作。格式: 1 字节 NUL + ASCII 头行(name/size/crc/type) + gzip(tar)。CRC = MD5("nradio_security_akxo1f\n" + gzip 内容)。

4.1 NR 查看/校验/解包

[NR 查看信息+校验]: 解析 .nr 头部字段、CRC 校验、列出 tar 成员。

[NR 严格校验]: 逐字节验证 CRC，报告是否被篡改。

[NR 解包到目录]: 解 gzip → 解 tar → 导出文件目录。

4.2 目录打包为 .nr

[目录打包为 .nr]: 目录 → GNU tar → gzip(mtime=0, OS=3) → 计算 MD5 CRC → 组装 NUL + 头行 + gzip。是解包的逆过程。

4.3 一键制作解锁备份

[★一键制作解锁备份]: 在原有 .nr 内容基础上注入 uci-defaults 脚本(99-nr-unlock)，设置以下三项:

· oem.custom.com = '' （取消 AT 页隐藏）

· firewall.ssh.target = ACCEPT （开启 SSH）

· adbd.service.enable = 1 （开启 ADB）

修改后重新打包并重新签名 CRC，产出可直接通过 Web 界面上传恢复的解锁备份文件。恢复后设备自动重启并执行 uci-defaults 脚本。

> *提示: 恢复 .nr 备份会强制删除 etc/config/firewall 和 etc/config/adbd，因此解锁脚本必须在 firewall/adbd 启动前执行（uci-defaults 机制保证）。*

第五章 页签③ 逆向打包

本页签提供解包流的逆向操作，从目录或分区文件重建归档/镜像/固件包。

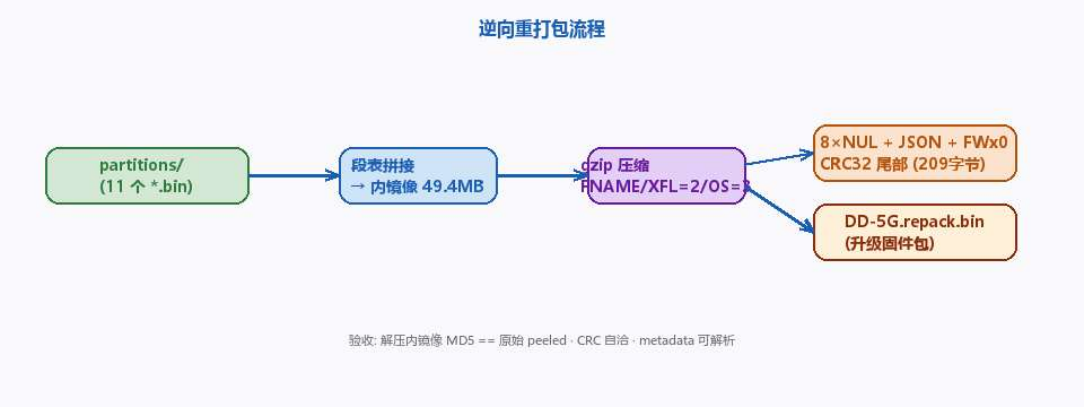


图 5-1 逆向重打包流程示意

5.1 目录→cpio / tar

[目录→cpio (+压缩)]: 纯 Python newc(070701) 生成器，4 字节对齐，含目录/文件/符号链接，末尾 TRAILER!!!，最后补 0 到 512 字节块边界（内核 initramfs 惯例）。可选 gzip/xz/lzma/bz2/zstd 压缩。

[目录→tar (+压缩)]: GNU tar 格式，保留目录/符号链接/权限，root:root，mtime=0。可选压缩。

5.2 文件→压缩包裹

[文件→压缩包裹]: 对已有文件做压缩，gzip 仿真设备风格(mtime=0, OS=Unix)，支持 xz/lzma/bzip2/zstd。

5.3 目录→JFFS2 / squashfs 镜像

[目录→JFFS2 镜像]: 调用外部 mkfs.jffs2 (mtd-utils)，需在 WSL 内执行。命令参考:

```
mkfs.jffs2 -r <目录> -o out.jffs2 -e 0x20000 -n
```

[目录→squashfs 镜像]: 调用外部 mksquashfs，可选 xz/gzip/lzma/zstd/lzo 压缩。

5.4 分区段拼接为整镜像

[分区段拼接为整镜像]: 按选择顺序拼接分区段文件，可选按擦除块(如 0x20000)用 0xFF 填充对齐（海思分段镜像惯例）。是切分区段的逆操作。

5.5 ★ 分区→fwtool 升级包（新增）

[★分区→fwtool 升级包]: 从一键解包产出的 partitions 目录(11 个 <name>.bin 分区文件)重建 OpenWrt fwtool 全量升级 .bin。这是"一键智能解包"的完整逆过程。



图 5-2 fwtool 尾部结构（209 字节）

打包流程:

1. 按 moses 段表顺序拼接 11 个分区为内镜像（各段已块对齐，无需额外填充）
2. gzip 压缩（FLG=0x08 带 FNAME=firmware.bin, XFL=2, OS=3）
3. 追加 fwtool 尾部: 8×NUL 头 + JSON + '\n' + 16 字节 'FWx0' 尾
4. CRC32 = ~CRC32(gzip 整包 + 8NUL + JSON 含 LF) & 0xFFFFFFFF
5. size 字段 = 8 + len(JSON) + 16 = 209

元数据来源优先级:

· 显式指定的 metadata.json 文件（GUI 可选 / CLI --metadata）

· 解包时自动保存的 peeled_0x0.bin.metadata.json

· 内置 moses 默认元数据（v24.1.1.101 / TD-RedCap / moses）

> *提示: 修改 system_rootfs 内容后，需先在 WSL 内用 mkfs.jffs2 重建 system.bin 段（尾段长度可变），再替换进 partitions 目录打包。其余 10 个固定段的长度不可改变，否则后续段偏移全部错位。*

验收标准: 新包 gzip 解压产物 == 原始内镜像（MD5 一致）； fwtool CRC 自洽；metadata 可解析；再切 11 段 MD5 与原分区一致。整包 MD5 因 deflate 流不同而不等是正常的——设备端只校验自身 CRC。

第六章 CLI 命令行模式

不启动 GUI，直接在命令行执行。语法:

```
python 固件解包工具.py --cli <子命令> [参数]
```

全部子命令:

子命令	参数	说明
scan	file	扫描识别 magic/尾部/gzip 成员
peel	file -o out [--offset N]	剥一层压缩
cpio	file -o out	解包 cpio newc
split	file --offset N --length L -o out	切分区段
fs	file -o out	解包文件系统
parts	file	找分区线索
probe	file	分段裸包内部探测
auto	file [-o out]	一键智能解包
nr-info	nr	NR 备份查看信息
nr-verify	nr	NR 备份严格校验
nr-unpack	nr [-d dir]	NR 备份解包
nr-pack	dir -o out	目录打包为 .nr
nr-unlock	nr [-o out] [--no-at]	一键制作解锁备份
pack-cpio	dir -o out [--compress gzip]	目录→cpio
pack-tar	dir -o out [--compress	目录→tar

	gzip]	
compress	file --format gzip [-o out]	文件→压缩包裹
mkfs	dir -o out --fs jffs2	目录→FS 镜像
join	seg1 seg2... -o out	分区拼接
pack-fwtool	partitions -o out	★ 分区→fwtool 升级包

示例 -- 从 partitions 目录重建 DD-5G 升级固件:

```
python 固件解包工具.py --cli pack-fwtool ^
  "解包工具 bare2.0\DD-5G_2.1.11_unpack\partitions" ^
  -o "DD-5G_2.1.11.repack.bin"
```

第七章 实战: DD-5G 固件解包→修改→重打包

步骤 1: 一键解包

GUI 操作:

1) 切到「固件解包」页签

2) 输入文件选 DD-5G_2.1.11.bin

3) 输出目录留空（自动创建 DD-5G_2.1.11_unpack\）

4) 点击「一键智能解包」

5) 等待日志出现「==== 完成: 一键智能解包 ====」

产出: partitions/(11 个分区.bin) + system_rootfs/ + initramfs/ + teeos_cpio.bin + kernel_Image.bin + initramfs_cpio.bin + peeled_0x0.bin + metadata.json + trailer.bin

步骤 2: 修改 rootfs

在 system_rootfs/ 目录内做所需修改（如编辑 etc/config/ 下配置文件、添加脚本等）。修改完成后需重建 system.bin:

```
wsl mkfs.jffs2 -r system_rootfs -o system.bin -e 0x20000 -n
```

将重建的 system.bin 替换到 partitions/ 目录。

步骤 3: 重打包为升级固件

GUI 操作:

1) 切到「逆向打包」页签

2) 源目录选 partitions/ 目录

3) 输出路径留空（自动命名为 DD-5G_2.1.11.repack.bin）

4) fwtool 元数据留空（自动复用解包时的 metadata.json）

5) 点击「★分区→fwtool 升级包」

6) 等待日志出现「==== 完成: 打包 fwtool 升级包 ====」

或用 CLI:

```
python 固件解包工具.py --cli pack-fwtool ^
  "解包工具 bare2.0\DD-5G_2.1.11_unpack\partitions" ^
  -o "DD-5G_2.1.11.repack.bin"
```

步骤 4: 验证

新包 gzip 解压后的内镜像与原始 peeled_0x0.bin 应 MD5 一致（未改分区内容时）。

工具自身函数可验证:

```
python -c "import importlib.util as u; s=u.spec_from_file_location("t","固件解包工具.py"); m=u.module_from_spec(s); s.loader.exec_module(m); p=open("DD-5G_2.1.11.repack.bin","rb").read(); i,un,f=m.decompress_one(p); print(f\"{f} inner={len(i)} unused={len(un)}\")
```

附录 A moses 段表参考

DD-5G (华为 Hi6352 RedCap) 全量升级包段布局，块大小 128K (0x20000):

序号	分区名	块数	字节长度	说明
1	xloader	4	0x80000	一级 bootloader
2	ptable	3	0x60000	分区表
3	mnvdload	4	0x80000	Modem NV 下载
4	uce	3	0x60000	UCE
5	hiboot	10	0x140000	二级 bootloader
6	dts	4	0x80000	设备树
7	m3image	4	0x80000	M3 镜像
8	teeos	8	0x100000	可信 OS
9	trusted_fw	3	0x60000	可信固件
10	kernel	50	0x640000	Linux 内核(含 initramfs)
11	system	尾段	可变	JFFS2 rootfs

段表来源: initramfs 内 upgrade.sh 的 fw_full_upgrade 行。skip/count 一律按十进制解析。*bk 后缀的备份段已自动合并。

内镜像总长 = 0x2F1A000 (49,389,568 字节)。gzip 压缩后约 41.6MB，加 209 字节尾部即为升级包。

附录 B 常见问题

Q: 双击 .cmd 后窗口闪退?

A: 检查 Python 是否在 PATH（python --version）; 检查 .py 与 .cmd 是否同目录; 在 CMD 中手动运行 python 固件解包工具.py 查看报错。

Q: jefferson 解包报错退出码 4294967295?

A: 这是 unsigned -1 的正常返回值。工具已自动处理: 调用前 rmtree 目标目录，设置 UTF-8 环境变量。只要日志显示解出了文件即算成功。

Q: 打包后的 .bin MD5 与原始不一致?

A: 这是正常的。gzip deflate 流不可能逐字节复现，但解压后的内镜像可以完全一致。验收标准是: 解压产物 MD5 == 原始，CRC 自洽，metadata 可解析，设备端能正常校验升级。

Q: 修改了 system_rootfs 后怎么打包?

A: 先在 WSL 内用 mkfs.jffs2 重建 system.bin，替换到 partitions/ 目录，再用「★分区→fwtool 升级包」打包。system 是尾段，长度可变; 其余 10 个固定段长度不可改。

Q: fwtool 元数据怎么填?

A: 留空即可，工具自动查找解包时保存的 peeled_0x0.bin.metadata.json。如需修改版本号/设备名，编辑该 JSON 后在 GUI 中手动指定路径，或 CLI 加 --metadata 参数。

Q: 支持 DD-5G 以外的固件吗?

A: fwtool 段表目前内置 moses (DD-5G) 布局。其他 OpenWrt fwtool 升级包可通过解包其 initramfs 内的 upgrade.sh 获取段表，在 FWTOOL_LAYOUTS 字典中添加后即可支持。